

Computer Vision 1A : Lab

22 septembre 2026

Table des matières

1	Preamble	5
1.1	Content	5
1.2	Staff	5
2	Optical imaging (2h)	7
2.1	Introduction	7
2.2	Experimental setup	7
2.3	Setup analysis	8
2.3.1	Starting uEye Cockpit	8
2.3.2	Analysing the 25 mm focal length objective	8
2.3.3	Analysing the 25 mm focal length objective with an extension tube	9
3	Cognex In-Sight Vision Suite (ISVS) conventionnel : Apprentissage	11
3.1	Dispositif expérimental	11
3.2	Prise en main du système	11
3.2.1	Démarrer ISVS	11
3.2.2	Configurer le système (si besoin)	11
3.2.3	Travailler sur une caméra réelle	13
3.2.4	Travailler sur une caméra émulée	14
3.2.5	Mode de programmation Feuille de calcul (Tableur)	14
3.2.6	Comprendre les fonctions ou outils	16
3.3	Application de contrôle d'aspect	16
3.3.1	Définition du repère de référence pour le recalage des outils	16
3.3.2	Méthode utilisant la segmentation par seuillage	17
3.4	Application de contrôle dimensionnel	18
3.4.1	Étalonnage	18
3.4.2	Définition du repère de référence pour le recalage	19
3.4.3	Méthode utilisant la segmentation par détection de contours	19
4	Cognex In-Sight Vision Suite (ISVS) - Edge Learning : Apprentissage	21
4.1	Introduction	21
4.2	Programmer avec ViDi EL	21
5	OpenCV with Python (8h)	23
5.1	Experimental setup	23
5.2	Reading a video stream from the camera	23
5.3	Reading, processing and analysis a single image	24
5.4	Reading, processing and analysis a video stream from file	26
5.5	Color paper sorting application	27
5.6	Static autofocus application	29

Chapitre 1

Preamble

1.1 Content

The booklet contains the following practical work :

- the understanding of optical imaging,
- the learning of the In-Sight Vision Suite (ISVS) from Cognex,
- the learning of the OpenCV library along with Python.

After learning, two application sessions will allow to test and assess knowledge of the subjects.

The booklet is available on Moodle, no paper version will be supplied.

1.2 Staff

- Sounkalo Dembélé (MCU @ UMLP, Head of the class) : lectures, tutorials, labs

Chapitre 2

Optical imaging (2h)

2.1 Introduction

The pedagogical objective of this practical work session is the understanding of the conventional optical imaging system. At the end, this should enable to design this kind of system for a given application, more particularly an industrial application.

2.2 Experimental setup

The setup corresponds to a **modular system** ; it includes (figure 2.1) :

- a backlight,
- a frontlight,
- a 2D camera equipped with an imaging objective along with, or not, an extension tube,
- a computer running Windows OS and a software enabling camera configuration, image acquisition and processing.

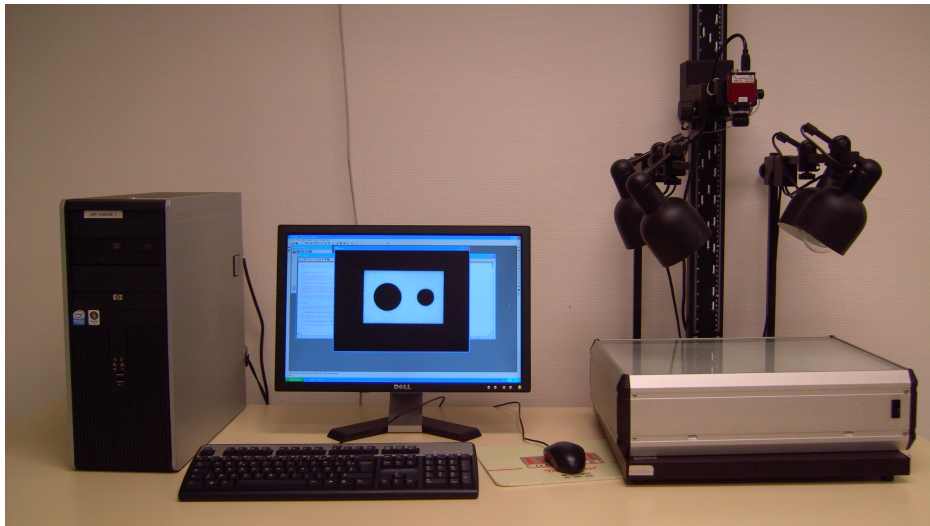


FIGURE 2.1 – The experimental setup with the IDS camera.

The 2D camera (IDS uEyeSE) specifications are :

- 1/2" CMOS color sensor (diagonal : 8 mm),
- 2048 x 1536 pixels full frame size,
- 8 bits depth,
- rolling shutter acquisition,

- 11 Hz full frame FPS,
- USB2.0 output.

The specifications of the objective (equivalent to Edmund Optics 25 mm UC series Fixes Focal Length Lens) are :

- C mount,
- 25 mm focal length,
- 1.6 to 16 aperture,
- 200 mm minimum working distance,
- infinite maximum working distance.

The specifications of the extension tube are :

- C mount,
- 5 to 40 mm length.

The camera is mounted on a support that enables the modification of the working distance for the focusing of the system.

2.3 Setup analysis

The **uEye Cockpit** software from IDS will be used to characterize the optical imaging system. Let us recall that the focusing of that kind of imaging system includes the setting of : the working distance, the image (to-lens) distance, the diaphragm opening, the gain and offset of the amplifier, the FPS, etc.

2.3.1 Starting uEye Cockpit

Run **uEye Cockpit**, if required check `Démarrer en mode expert`, click on **Monochrome** the **Lecture**.

It may be necessary to synchronize the camera with the screen, by setting the FPS to a sub-multiple of the screen frequency (usually greater than 60 Hz), for example to 20.

The images may appear distorted, but this is not a big issue for our task.

Use the **Configuration** mode to visualize and analysis the effects of the imaging parameters change.

2.3.2 Analysing the 25 mm focal length objective

From the image of a flat object obtained with the front lighting system and a graduate ruler to estimate :

1. the minimal working distance, the minimal field-of-view, the maximal magnification, the best achievable resolution
2. the maximal working distance
3. the depth-of-field.

2.3.3 Analysing the 25 mm focal length objective with an extension tube

Add an extension tube to the system and recalculate the parameters : minimal working distance, minimal field-of-view, maximal magnification, best achievable resolution, maximal working distance, depth-of-field.

Manipulate the components with caution !

To understand the geometrical constraints introduced by the extension tube acquire the images of small watch parts, particularly with the height greater than the depth-of-field.

Make the link with the PhD of Julien Bert, defended in 2007 :

1. the properties of optical microscopes (chapter 2),
2. the construction of a global sharp image by concatenating local sharp images (cf. numerical microscope from Keyence),
3. the construction of panorama or mosaïc (chapitre 4),
4. the access to the third dimension from lateral views (chapter 5 and Percipio Robotics machines).

Chapitre 3

Cognex In-Sight Vision Suite (ISVS) conventionnel : Apprentissage

3.1 Dispositif expérimental

Le dispositif qui sera utilisé pendant ces travaux comprend deux caméras **intelligentes**, In-Sight 2802C, qui vont avec le logiciel In-Sight Vision Suite (ISVS), tous de la société Cognex, un plateau tournant à vitesse réglable où sont disposés les objets/pièces à traiter, et un PC (figure 3.1).

In-Sight Vision Suite est une suite logicielle pour le développement et le déploiement d'applications de vision. Il comprend un double module :

- le module **Tableur** (Spreadsheet) permet d'accéder à une bibliothèque bien fournie de fonctions pour le traitement, l'analyse et l'interprétation des images,
- le module **EasyBuilder** est limité aux principales fonctions mais permet, en principe, de développer plus rapidement les applications.

3.2 Prise en main du système

3.2.1 Démarrer ISVS

Le démarrage du logiciel conduit à l'écran d'accueil (figure 3.2) avec les rubriques suivantes :

1. le nom du poste de travail, les symboles **Roue**, **Flèche circulaire** (Actualiser les périphériques) et **Aide** (Aide)
2. la liste des caméras réelles et émulées.

3.2.2 Configurer le système (si besoin)

Cliquer sur **Roue / Paramètres du système** donne un écran où on peut modifier la langue (**Inutile de changer en Français si les rubriques sont déjà en Français**).

Cliquer sur

Roue / Détection de périphérique / Configurer manuellement les périphériques donne un écran où on peut configurer manuellement les périphériques (inutile pour nous).

Cliquer sur

Roue / Détection de périphérique / Paramètres de sous-réseau distant

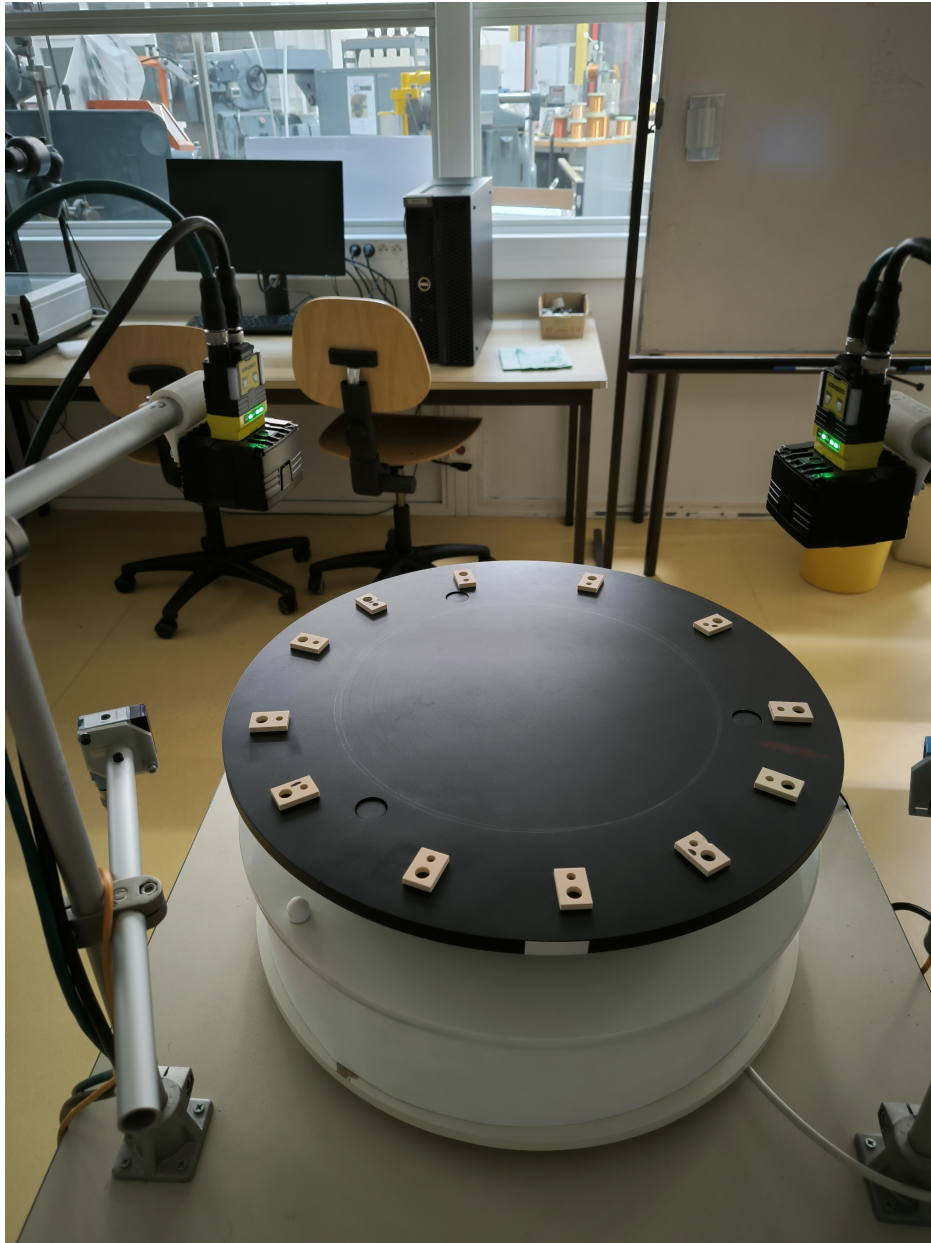


FIGURE 3.1 – Le dispositif expérimental avec caméra Cognex.

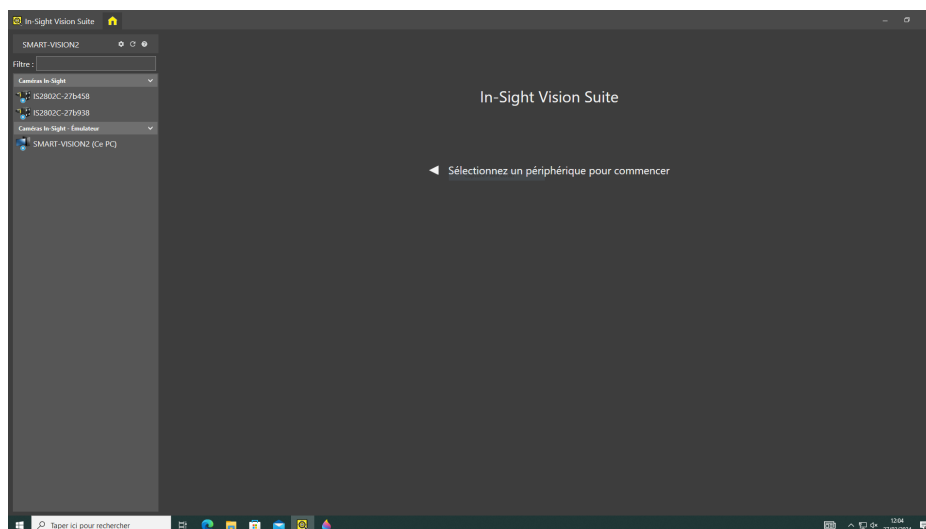


FIGURE 3.2 – Copie écran d'accueil.

Poste	Offline Programming Reference	Offline Programming Key
Vision-1	8f4d8bf6	c341d5b2
Vision-2	8f4d8bf6	bb49f132
Vision-3	8f4d8bf6	37556f76
Vision-4	8f4d8bf6	3f5d73f6
Vision-5	8f4d8bf6	b3b1cdb2

TABLE 3.1 – Les codes pour les émulateurs ISVS.

donne un écran où on peut configurer un sous-réseau (inutile pour nous).

Cliquer sur

Roue / Détection de périphérique / Paramètres réseau du PC

donne un écran où on peut configurer les paramètres réseau du PC hôte (inutile pour nous).

Cliquer sur

Roue / Détection de périphérique / Découvrir les périphériques mal configurés permet de générer la liste des caméras réelles connectées (utile pour démarrer la configuration des caméras).

Cliquer sur

Roue / Émulateur

donne un écran où on peut configurer l'émulateur de caméra (très utile) : attendre un peu avant la caméra émulée n'apparaisse dans la liste des caméras In-Sight.

Pour mémoire, les codes des émulateurs sont donnés dans le tableau 3.1 :

3.2.3 Travailler sur une caméra réelle

On notera que Cognex appelle une caméra un **capteur**.

Cliquer sur la Caméra ISC2802C de son choix donne un écran (figure 3.3) où on trouve :

1. les informations sur la caméras (nom, adresse IP, adresse MAC, etc.),
2. les liens d'accès aux modes de programmation **EasyBuilder** et **Feuille de calcul** et l'ouverture de l'IHM,
3. une liste d'utilitaires dont la Mise à jour du firmware, Redémarrage, Configuration de caméra, Paramètres de démarrage, Paramètres de réseau.

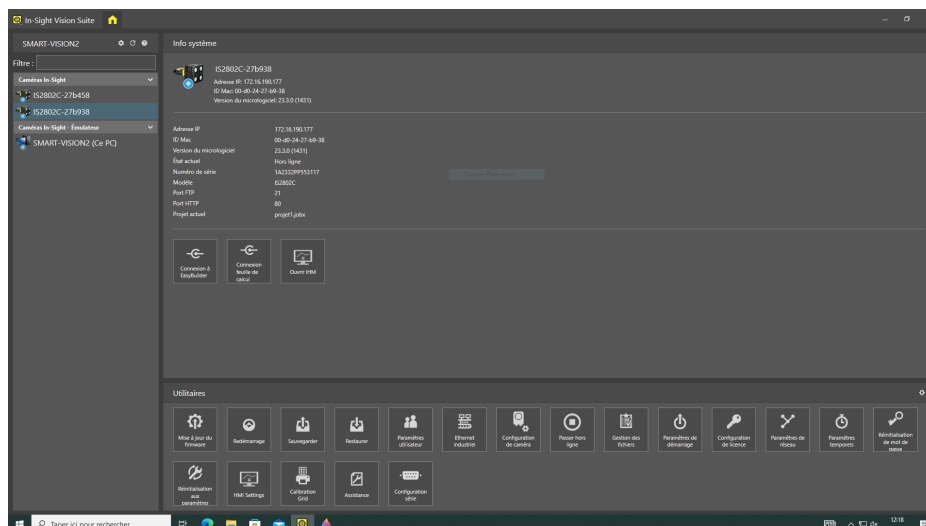


FIGURE 3.3 – Copie écran d'accueil d'une caméra réelle.

3.2.4 Travailler sur une caméra émulée

Cliquer sur la **Caméra émulée** donne un écran où on trouve :

1. les informations sur la caméras (nom, adresse IP locale, adresse MAC, etc.),
2. les liens d'accès aux modes de programmation **EasyBuilder** et **Feuille de calcul** et l'ouverture de l'IHM,

3.2.5 Mode de programmation Feuille de calcul (Tableur)

Cliquer sur **Connexion feuille de calcul** donne un écran (figure 3.4) où on trouve :

- les menus **Fichiers (F)**, **Capteur** et **Afficher (V)**,
- la liste des raccourcis
- la zone de programmation et d'affichage de l'image de travail,
- la zone **FilmStrip** des images de travail avec l'affichage des images, leur dossier de stockage (en bas à gauche), le bouton d'enregistrement (en bas à gauche), les boutons de lecture des images (en bas à droite),
- la boîte à outils avec les onglets **Fonctions** et **Acquisition**

La barre d'outils permet de configurer la zone de travail, avec notamment

- **Modifier la division (Shift+F7)** pour avoir l'image de travail et le code dans deux fenêtres distinctes,
- **Afficher la grille (Shift+F6)** pour avoir la feuille de calcul en superposition sur l'image,
- **Afficher les graphiques (Ctrl+Shift+G)** pour afficher les graphiques des fonctions sur l'image,
- etc.

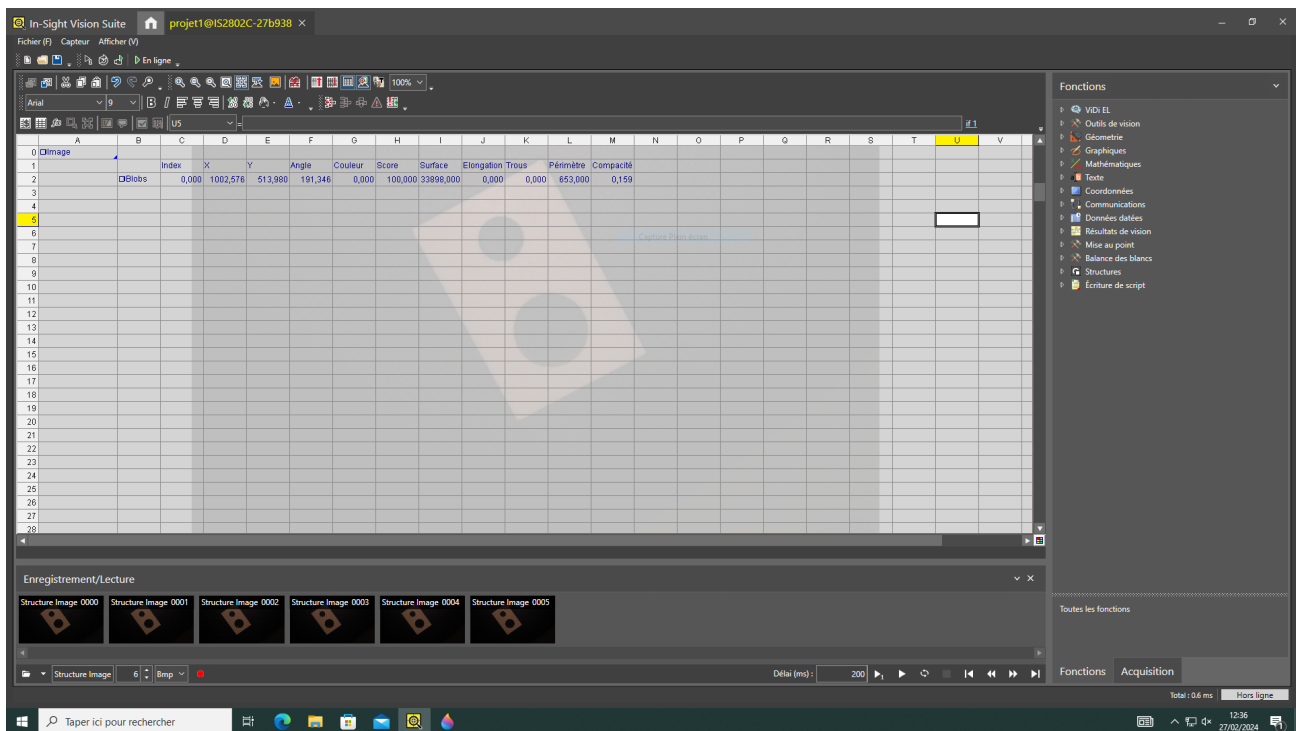


FIGURE 3.4 – Copie écran d'accueil du Tableur.

Acquérir manuellement des images sur caméra réelle

Démarrer Tableur : la cellule A0 contient la fonction/outil `AcquireImage` qui retourne une image.

Cliquer sur Vidéo directe.

Cliquer sur l'onglet Acquisition (ou A0) pour configurer la caméra, notamment, activer le paramètre Mise au point auto.

Pour acquérir une image : choisir le Repertoire Enregistrement/Lecture, cliquer sur Bouton rouge (Enregistrement), cliquer sur Déclencheur (F5) (dès fois cela ne marche pas!), dans ce cas cliquer sur l'icône Enregistrer image.

Acquérir automatiquement des images sur une caméra réelle

Démarrer Tableur : la cellule A0 contient la fonction/outil `AcquireImage` qui retourne une image.

Cliquer sur l'onglet Acquisition (ou A0) pour configurer la caméra, notamment

1. Déclenchement, choisir Simple
2. activer Mise au point automatique

Cliquer sur l'icône Passer en ligne (CTRL+F8)

Comprendre le repère image

Dans ISVS le repère image est tel que l'axe X est horizontal et orienté vers la droite et l'axe Y est vertical et orienté vers le bas (comme le standard). D'autre part, un point est repéré outre par sa position X , sa position Y , mais également son angle par rapport à l'axe X (*Angle*).

Il y a deux manières de référencer un point :

- absolue (grille avec symbole \$), dans ce cas la cellule associée ne change pas quand elle est copiée dans une nouvelle position,
- relative (grille sans symbole \$), dans le cas contraire.

3.2.6 Comprendre les fonctions ou outils

L'onglet **Fonctions** comprend onze familles d'outils :

1. **Outils de vision** regroupe les outils de traitement d'images et de vision,
2. **Géométrie** regroupe les outils de traitement géométriques des données
3. **Graphiques** regroupe les outils de création d'interfaces,
4. **Mathématiques** regroupe les outils mathématiques et informatiques,
5. **Texte** regroupe les outils de traitement de caractères,
6. **Coordonnées** regroupe les outils d'échelle et de manipulation de repère des objets,
7. **Communications** regroupe les outils d'échange de données sur le réseau Ethernet,
8. **Données datées** regroupe les outils de manipulation des données,
9. **Résultats de vision** regroupe les outils de récupération des données retournées par les outils de vision,
10. **Balance des blancs** regroupe les outils permettant d'obtenir et définir les valeurs de la balance des blancs,
11. **Structures** regroupe les fonctions de création de structures géométriques spécifiques,
12. **Écriture de script** permet d'écrire un script JavaScript.

3.3 Application de contrôle d'aspect

On se propose de réaliser l'inspection de l'aspect des poutres disponibles sur le plateau rotatif : il s'agit de détecter la présence de défaut(s) d'aspect, c'est-à-dire de taches, à la surface exposée desdites poutres.

3.3.1 Définition du repère de référence pour le recalage des outils

Dans le contexte de la production, le recalage est une opération préalable à tout traitement : il permet de définir un repère de référence à partir d'une région modèle dans l'image et d'adapter les régions d'intérêt (où sont réalisées les traitements et mesures) par rapport à ce repère. Il permet aux traitements d'être robustes à la position et à l'orientation (un peu moins !) des objets.

Cliquez sur l'image de *l'objet sans défaut* ou *étalon* ou encore *gabarit*.

Le repère de référence se définit depuis l'image du gabarit en deux étapes : *entraînement* et *application*.

Dans l'étape d'entraînement, le système apprend le modèle se trouvant dans la région de l'image indiquée :

Outils de vision > Correspondance de modèle > TrainPatMaxRedLine
avec les changements de paramètres suivants :

- **Région du modèle** qui doit ressembler à l'exemple de la figure 3.5,
- **Seuil de caractéristique** éventuellement ajustée,

— Réutiliser l'image d'apprentissage activée.

Dans l'étape d'application, le système recherche le modèle appris grâce à la fonction **Outils de vision > Correspondance de modèle > FindPatMaxRedLine** avec les changements de paramètres suivants :

1. avec **Région de recherche** qui doit correspondre au champ de vision de la caméra diminuée d'environ la moitié de taille de la région qui a servi à entraîner le modèle,
2. avec **Modèle** qui pointe (en absolu) sur la cellule **Patterns** de l'étape d'entraînement,
3. **Nombre à chercher** à 1,
4. **Seuil d'acceptation** ajustée,
5. **Rech. Tolérances** ajustées pour la rotation et l'échelle.

Dans la suite, pour recalibrer une région (ou une image), on fait pointer son repère (en *absolu*) vers le repère de référence en sélectionnant les données *Ligne*, *Colonne* et *Angle*.

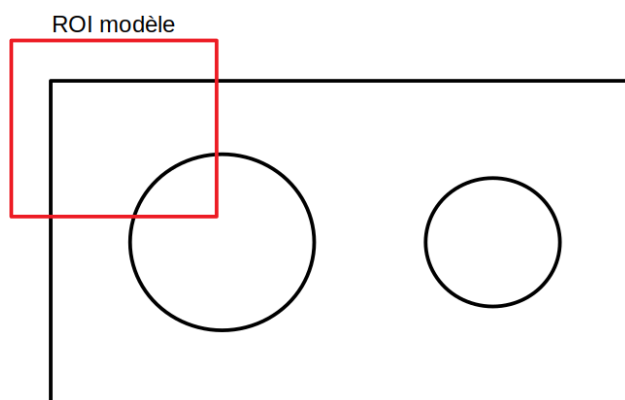


FIGURE 3.5 – Région modèle pour le recalage.

3.3.2 Méthode utilisant la segmentation par seuillage

Dans ISVS la fonction de segmentation est **Système de vision > Blob > DetectBlobs** (sachant qu'un blob est un nuage de points et correspond pour Cognex à une région). Elle comporte la binarisation des image, le filtrage des régions, et exige une analyse au préalable de l'image pour pouvoir choisir les arguments et interpréter les mesures associées que l'on trouve dans **Résultats de vision > Blobs**.

Recherchez les défauts d'aspect (taches) à partir des images de travail :

1. segmentez les images avec **Région externe** à 0, **Nombre à classer** à 0, **Couleur** à indifférent et **Fond** à blanc (**Système de vision > Blob > DetectBlobs**¹) ;
2. calculez le nombre de régions (blobs) de l'image (**Résultats de vision > Blobs > GetN-Found**) ;
3. trouvez et codez la formule permettant de calculer le nombre de défauts ;
4. affichez le nombre de défauts sur l'image (**Texte > Chaîne > Stringf** pour convertir le nombre en string et **Graphiques > Image > PlotString** pour afficher le nombre converti en string) ;

1. Attention : il faut adapter les paramètres, notamment les nombres minimal et maximal de pixels

5. affichez l'état bon ou mauvais de l'objet, dans la même couleur ;
6. affichez l'état bon ou mauvais de l'objet, dans des couleurs ; différentes².

Utilisation de `Stringf : Stringf("%c", N)` avec `N` le numéro de la cellule ciblée contenant un, `Stringf("%.nf", N)` avec `n` le nombre de chiffre(s) après la virgule et `N` le numéro de la cellule ciblée.

Testez votre code sur la caméra de travail.

Avant de réaliser la segmentation il est possible de pré-traiter l'image pour faciliter son interprétation. Typiquement cela comprend :

1. le calcul de l'histogramme pour déterminer le meilleur seuil de binarisation : **Outils de vision > Histogramme > ExtractHistogram** ;
2. la binarisation de l'image : **Outils de vision > Image > Filter** en utilisant le paramètre **Binariser** et le seuil trouvé précédemment ;
3. l'ouverture pour nettoyer les nuages de points : **Outils de vision > Image > Filter** avec le paramètre **Ouverture**
4. la fermeture pour lisser les contours des nuages : **Outils de vision > Image > Filter** avec le paramètre **Fermeture**
5. ...

Appliquez ces traitements aux images de travail.

3.4 Application de contrôle dimensionnel

On se propose de réaliser l'inspection des dimensions des poutres disponibles sur le plateau rotatif : il s'agit de vérifier la conformité de la largeur, du diamètre du grand trou et de la position du grand trou par rapport à coté le plus proche. Les cotes associées et leurs valeurs de référence sont indiquées à la figure 3.6).

3.4.1 Étalonnage

L'unité de base des mesures en vision est le pixel, pour les avoir en unités métriques (mm ici), il faut étalonner (au moins partiellement) le système, c'est-à-dire calculer le facteur d'échelle c'est-à-dire le rapport entre pixel et mm (en **pixel/mm** ici).

Procédez comme suit pour étalonner le système :

1. Pointez sur l'image du gabarit ;
2. Calculez le contour extérieur du grand cercle par détection de contour et ajustement : **Outils de vision > Bord > FindCircle** (faire attention au seuil de binarisation qu'il faudra au besoin adapter!!!) ;
3. Calculez le contour extérieur du petit cercle par détection de contour et ajustement : **Outils de vision > Bord > FindCircle** (faire attention au seuil de binarisation qu'il faudra au besoin adapter!!!) ;

2. Solution1 : afficher l'état bon avec une séquence contenant un `if(...)` et un `PlotString`, idem avec l'état mauvais ; solution 2 : pour afficher l'état bon décocher **Affichage**, cliquer sur l'icône **fx**, insérer l'expression avec la cellule du `if`, utiliser `PlotString`, idem pour mauvais

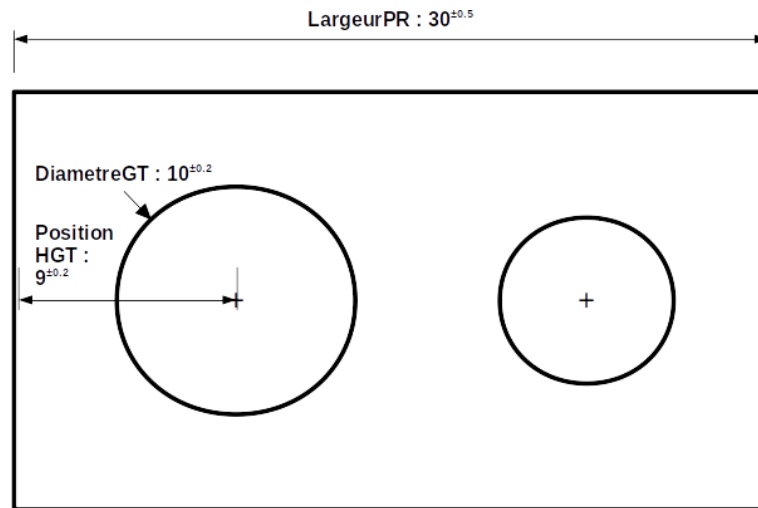


FIGURE 3.6 – Les cotes (en mm) à mesurer sur chaque poutre.

4. Calculez la valeur de l'entraxe en pixels, c'est-à-dire la distance entre les centres des deux cercles : **Géométrie > Mesure > PointToPoint** ;
5. Calculez le rapport pixel-mm à partir de la valeur de l'entraxe en pixels sachant que sa valeur en mm est de **14,1** ;
6. Désactivez la cellule.

3.4.2 Définition du repère de référence pour le recalage

Déterminez le repère de référence afin de pouvoir recalcr les outils (fonctions de vision).

3.4.3 Méthode utilisant la segmentation par détection de contours

La détection de contours permet de calculer les variations de valeurs significatives des images (en anglais contour se dit edge). Elle se chevauche avec la segmentation pour les contours fermés, mais tous les contours ne sont pas forcément fermés.

Dans ISVS les fonctions de détection de contours sont dans **Outils de vision > Bord** et les fonctions de mesure associées sont dans **Géométrie > Mesure** et **Résultats de vision**.

Remarque importante :

La flèche dans la définition de la ROI de recherche indique le sens de calcul du contour, elle doit être horizontale pour un contour vertical (par défaut) et verticale pour un contour horizontal (faire tourner la ROI).

Contrôlez (mesurez et affichez sur l'image) le diamètre du grand trou en utilisant la fonction **Outils de vision > Bord > FindCircle**, **Résultats de vision > CircleFit > GetRadius**.

Contrôlez (mesurez et affichez sur l'image) la cote PositionHGT en utilisant la fonction **Outils de vision > Bord > FindLine**, la fonction précédente et la fonction **Géométrie > Mesure > PointToLine**.

Contrôlez (mesurez et affichez sur l'image) la cote LargeurPR en utilisant la fonction **Outils de vision > Bord > FindLine**, la fonction précédente et la fonction **Outils de vision >**

Géometrie > Mesure > PointToLine.

La détection de contours de ISVS est fondée sur la fonction **Outils de vision > Image > Filter** où l'argument Opération a la valeur Amplitude contours. Cette fonction a déjà été étudiée.

Testez votre code sur la caméra.

Chapitre 4

Cognex In-Sight Vision Suite (ISVS) - Edge Learning : Apprentissage

4.1 Introduction

Outre les fonctions de traitement d'images conventionnel, ISVS inclut des fonctions à base de modèles de réseaux de neurones pré-entraînés pour résoudre trois tâches : la classification permettant de catégoriser des images, la lecture de caractères dans les images, et la segmentation d'images.

Les fonctions sont regroupées sous le terme **ViDi EL** qui doit correspondre à peu près à "Vision Design Edge Learning". *Edge Learning* correspond au fait d'interpréter les images au plus près du capteur. Cela implique l'embarquement du modèle de réseau de neurones, qui doit de ce fait être plus léger et en général pré-entraîné. L'utilisateur doit juste l'affiner (ou le personnaliser) pour résoudre son problème.

Le concept de Edge learning fait partie du concept de *Transfer learning* qui consiste à transférer la connaissance sur des objets à d'autres objets similaires.

4.2 Programmer avec ViDi EL

ViDi EL permet d'affiner trois modèles de réseaux de neurones, déjà entraînés :

1. **ViDiELClassify** pour faire de la classification d'images,
2. **ViDiELRead** pour faire de la reconnaissance de caractères dans les images,
3. **ViDiELSegment** pour faire de la segmentation d'images.

Chaque fonction (basée sur un modèle de réseaux) comprend trois onglets :

1. **Général** permettant de définir la source d'images, le repère¹, la région d'intérêt², la définition de la tâche³, le choix de l'affichage des données sur l'image ciblée,
2. **Apprentissage** permettant de définir les classes et étiqueter, ou non, successivement les images de la base d'entraînement⁴,
3. **Images** permettant de visualiser le résultat de l'entraînement du réseau.

La procédure d'entraînement de chaque modèle (via la fonction associée) est le suivant :

-
1. prendre le repère de référence si cela est nécessaire
 2. prendre une région incluant l'image
 3. entraînement, test, validation ou production
 4. étiqueter une image signifie l'associer à une classe

1. démarrer la fonction ;
2. paramétrer correctement l'onglet **Général**, cocher la case **Collecter les exemples** ;
3. déclarer les classes dans l'onglet **Apprentissage** ;
4. cliquer OK (alors doit apparaître les classes et les scores associés dans **Tableur**) ;
5. faire la lecture des images d'entraînement en commençant par la première ;
6. re-démarrer la fonction ;
7. dans l'onglet **Apprentissage** étiqueter (ou non) successivement les images de la base d'images ;
8. analyser les résultats dans l'onglet **Images**
9. cliquer OK.

Alors, le réseau est prêt pour le test, l'évaluation ou la production, selon la procédure suivante :

1. décocher la case **Collecter les exemples** ;
2. en mode hors-ligne, double cliquer sur chaque image individuelle ou cliquer sur **Lecture**, et en mode live, cliquer sur **Passer en ligne** ;
3. analyser les scores calculés et l'affichage de la classe de score le plus élevé sur l'image si l'affichage des entrées et sorties graphiques avaient été choisie dans **Général**.

Recommandation de Cognex :

Pour l'entraînement d'un modèle, la société recommande :

1. au minimum 5 à 7 images par classe,
2. au minimum 10% de différence entre les classes.

Chapitre 5

OpenCV with Python (8h)

The purpose of this part is the learning of OpenCV using Python. No report will be requested or mark be attributed.

5.1 Experimental setup

The modular imaging system, formerly described in the chapter dealing with optical imaging is used.

Procedure for running Spyder : run directly from the shortcut on the desktop.

Procedure for running Spyder with Anaconda :

1. run Anaconda from the Applications (may be long ...)
2. clic the "configuration icon"
3. cli "Interpréteur Python" and select
C:/Program Files/Python312/python.exe
4. clic OK

5.2 Reading a video stream from the camera

Copy, analyze and test the following code for reading a video stream until SPACE is clicked to stop :

```
import cv2

## Reading a video stream

# Configure video stream source: 0 for the embedded cam, 4 for the additional cam on
  this laptop
cam = cv2.VideoCapture(0, cv2.CAP_V4L2)
# cam = cv2.VideoCapture('paperAppr.mp4')
if (not cam.isOpened()):
    print ("Impossible to read the camera !")

# Display stream until clic on SPACE when mouse pointer in video display
while (True):
    ret, frame = cam.read()
    if ret:
        # print(f"Frame dimensions: {frame.shape}")
```

```

    cv2.imshow("video", frame)
    if cv2.waitKey(2)>=27:
        break
else:
    print("Error: Could not read a frame from the video stream.")
# Deallocate memory
cam.release()
cv2.destroyAllWindows()

```

Listing 5.1 – Python Code for reading a video stream

5.3 Reading, processing and analysis a single image

Consider the following code, that read, processes and analyzes the image `paperAppr.png` (figure 5.1).

Analyze the code then run it : during image display with `imshow`, the value of pixel under cursor is displayed bottom left.

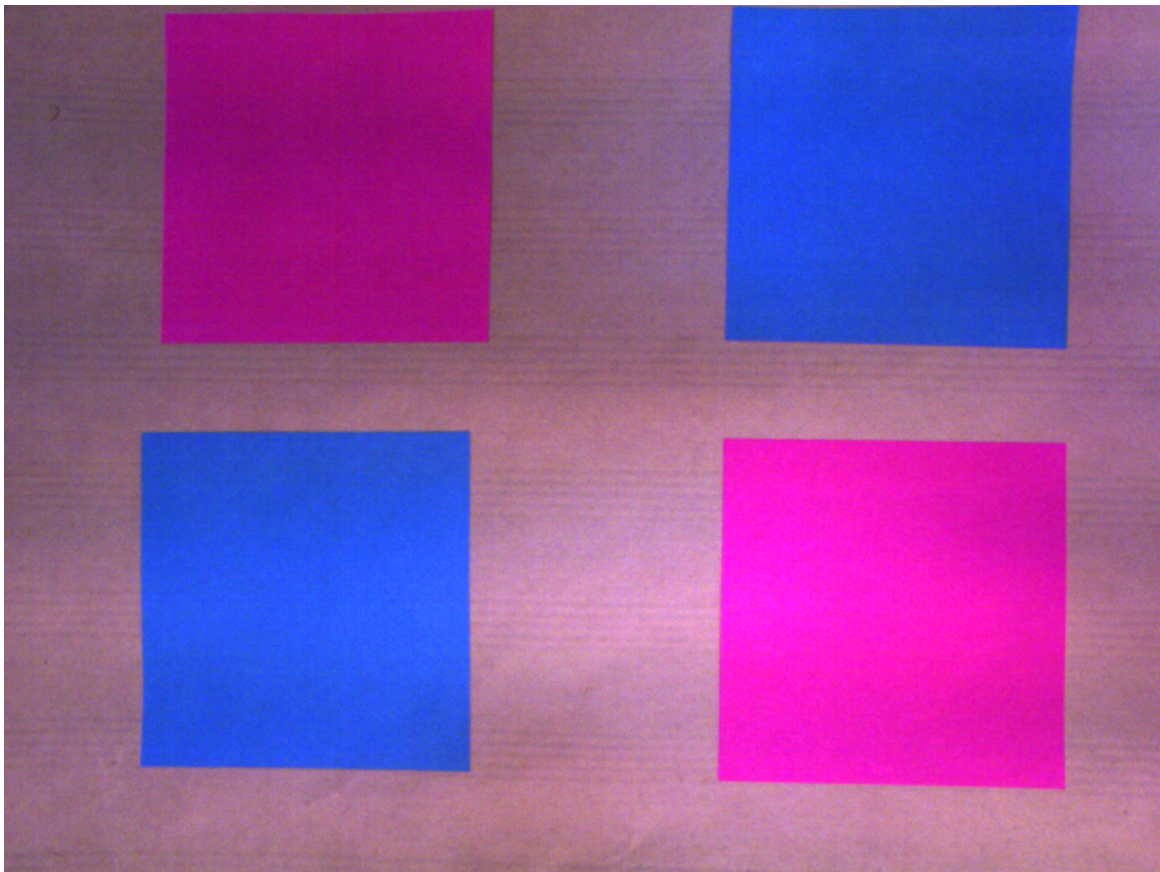


FIGURE 5.1 – Color image for testing.

```

"""
VISION 2D:
Training code
"""

```

```

import cv2

"""
Reading a single image
"""
# Read and display BGR image
bgr = cv2.imread ( './paperAppr.png', cv2.IMREAD_UNCHANGED )
if (not bgr.data):
    print ('Impossible to read image !')
cv2.imshow("RGB image", bgr)
cv2.waitKey(0)

# Convert BGR into HSV and display
hsv = cv2.cvtColor(bgr, cv2.COLOR_BGR2HSV)
cv2.imshow("HSV image", hsv)
cv2.waitKey(0)

# Extract hue and display
h,s,v = cv2.split(hsv)
cv2.imshow("Hue image", h)
cv2.waitKey(0)

# Convert BGR into GRAYSCALE and display
gray = cv2.cvtColor(bgr, cv2.COLOR_BGR2GRAY)
cv2.imshow("Gray image", gray)
cv2.waitKey(0)

# Display grayscale image with a color map
clr = cv2.applyColorMap( gray, cv2.COLORMAP_JET )
cv2.imshow("Color map image", clr)
cv2.waitKey(0)

# Global thresholding of GRAYSCALE and display
rv, bry = cv2.threshold(gray, 0,255, cv2.THRESH_BINARY_INV+cv2.THRESH_OTSU)
cv2.imshow("Binary image", bry)
cv2.waitKey(0)

# Local thresholding of GRAYSCALE and display
bry2 = cv2.adaptiveThreshold(gray, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,\
                             cv2.THRESH_BINARY, 11, 2)
cv2.imshow("Binary image 2", bry2)
cv2.waitKey(0)
cv2.destroyAllWindows()

```

Listing 5.2 – Python Code for reading a video stream

Complete the code to perform the following tasks :

1. erosion, dilatation, opening and closing of images, grayscale as well as global-thresholded binary images (functions `erode`, `dilate` et `morphologyEx`, respectively),
2. calculating the regions of binary image assuming a closed edge corresponds to a region (function `findContours`),
3. calculating and displaying bounding rectangles of regions (function `boundingRect`),
4. deblurring of grayscale image with rectangle kernel (function `blur`) then with gaussian kernel (function `GaussianBlur`),

5. enhancement of grayscale image with morphological gradient (`morphologyEx`), Sobel gradient (function `Sobel`) then with Canny gradient (function `Canny`).

5.4 Reading, processing and analysis a video stream from file

It is similar to reading a video stream from a camera except the source number is replaced with the file name including extension, inside single or double quotes.

5.5 Color paper sorting application

The purpose of this part is testing of your skills in computer vision through the use of OpenCV. It is requested to supply a report describing the solution and the corresponding code. They will be used for mark.

Some colored papers such that each paper has a unique uniform color are considered : pink, purple, blue, green, orange, yellow and white (figure 5.2).



FIGURE 5.2 – Reference image for color and sector calibration.

The papers move in a space structured in sectors numbered from top to bottom : 0 to 3.

The problem to solve consists in recognizing the color and position of the papers from their images recorded in a video (paperEval.avi).

The image `paperEval.png` is provided for manual calibration of the sectors and colors, assuming it is acquired in the same conditions as the video.

More precisely, the application should deliver per frame the following informations and save them in a file `resultats.txt` : identification of any object by its appearance number in the image, its color, the sector in which it (its center of inertia) is located (figure 5.3).

It is advised to organize the code as follows

- a function to manually calibrate the seven colors of the problem, i.e. to determine minimal and maximal hues of every color ;



FIGURE 5.3 – A frame illustrating application output.

- a function to manually calibrate the three sectors of the problem, i.e. to determine minimal and maximal locations of every sector ;
- a function to identify every paper by its order of appearance and calculate its color and sectors ;
- a function for display information : number, color, sector.

If you have time, you can use the `tkinter` library to create a GUI for the application.

5.6 Static autofocusing application

The purpose of this part is the testing of your skills in computer vision through the use of OpenCV. It is requested to supply a report describing the solution and the corresponding code. They will be used for mark.

A set of images of a daisy (*Bellis perennis*) pollen grain has been recorded with a scanning electron microscope at scanning speeds 5, 3, 2 and 1 while modifying the working distance d , i.e. the distance from the objective lens to the pollen grain (figure 5.4).

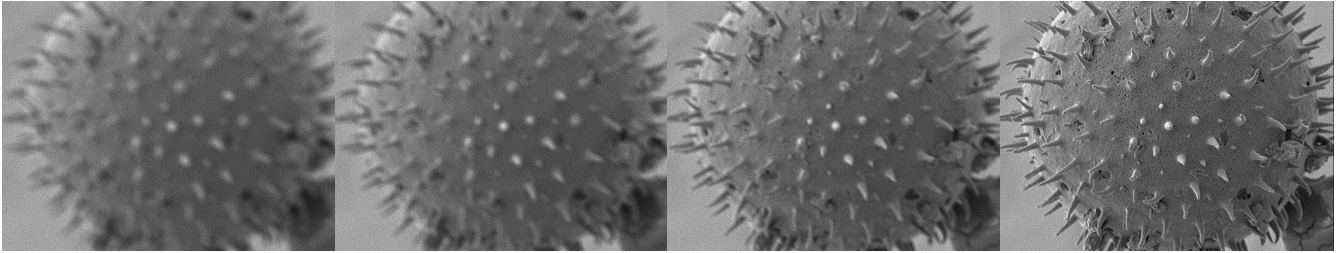


FIGURE 5.4 – Some images of the daisy (*Bellis perennis*) pollen grain at 5-scanning speed.

The stepsize of each motion is $10\mu\text{m}$. The minimum values of the working distance is 10 mm.

The problem to solve consists in finding the position of the *fine sharpest image* in the scanning speed imageset. This fine sharpest image is obtained by performing a quadratic curve fitting around the position of the *coarse sharpest image*, i.e., the sharpest image of the imageset. This coarse sharpest image is obtained using the derivative-based algorithm presented in the course (figure 5.5).

More precisely, the application should deliver per imageset (the input is the path of the imageset) the coarse sharpest image, its focus value and its position and displays it, and the fine sharpest images position. We advise to use normalized variance as focus measurement method.

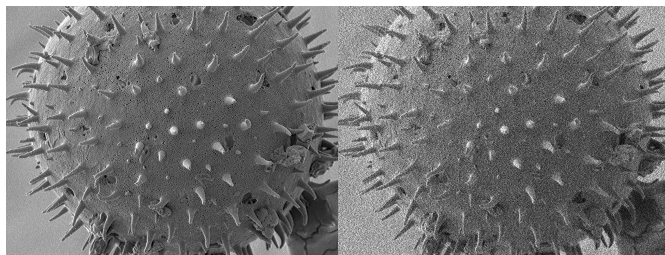


FIGURE 5.5 – Coarse sharpest image in scanning speeds 5, 3, 2, 1 imagesets.

It is advised to organize the code as follows

- a function to calculate the focus;
- a function to find the coarse sharpest image, its focus value and its position;
- a function to display the coarse sharpest image;
- a function to estimate the fine sharpest image position.

If you have time, you can use the `tkinter` library to create a GUI for the application.

If the method fails to find the maximum, by getting stuck in a local maximum, try a stochastic solution by randomly modify the decreasing factor, or any other solution of your choice.